

Artificial Intelligent LAB -6

Natural Language Processing (NLP)

What is NLP?

Natural Language Processing (NLP) is a subfield of Artificial Intelligence (AI) that enables computers to understand, interpret, and generate human language. It bridges the gap between human communication and computer understanding.

It combines **linguistics**, **computer science**, and **machine learning** to process and analyze large volumes of natural language data (text or speech).

Why Do We Need NLP?

Humans communicate using languages like English, Mandarin, or Urdu—but computers only understand binary or structured data. NLP enables:

- Interaction with computers in natural language.
 - Automation of repetitive language-based tasks.
 - Extraction of insights from unstructured text (e.g., social media, emails).
 - Enhancing user experiences in applications like chatbots, translators, and virtual assistants.
-

Significance of NLP

NLP is used in nearly every industry:

- **Healthcare:** analyze patient notes, assist diagnostics.
- **Business:** chatbots, sentiment analysis, automated email sorting.
- **Education:** grammar correction, automated essay scoring.
- **Law:** legal document summarization, contract analysis.
- **Robotics:** enable verbal commands and real-time interaction.

Key benefits include:

- Improved efficiency through automation.
 - Better decision-making using data extracted from unstructured text.
 - Human-like interactions between users and systems.
-

Code 1:

Text Similarity using Word Embeddings

```
% Clear previous variables and console
clear; clc;

% Define two English sentences for comparison
text1 = "The quick brown fox jumps over the lazy dog.";
text2 = "A fast brown animal leaps above a sleeping dog.";

% Tokenize the text (split into words, remove punctuation)
documents = tokenizedDocument([text1, text2]);

% Load pre-trained FastText word embeddings (if not downloaded, MATLAB will prompt)
emb = fastTextWordEmbedding;

% Extract unique vocabulary (words) from each sentence
words1 = string(documents(1).Vocabulary);
words2 = string(documents(2).Vocabulary);

% Filter out words that are not present in the embedding vocabulary
words1 = words1(isVocabularyWord(emb, words1));
words2 = words2(isVocabularyWord(emb, words2));

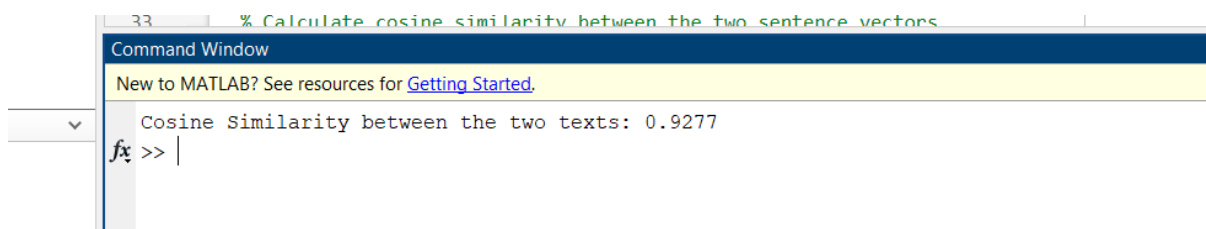
% Convert each word to its corresponding vector representation
vecs1 = word2vec(emb, words1);
vecs2 = word2vec(emb, words2);

% Compute the average (mean) word vector to represent the whole sentence
vec1 = mean(vecs1, 1);
vec2 = mean(vecs2, 1);

% Calculate cosine similarity between the two sentence vectors
similarity = dot(vec1, vec2) / (norm(vec1) * norm(vec2));

% Display the similarity result
fprintf("Cosine Similarity between the two texts: %.4f\n", similarity);
```

This section calculates the **semantic similarity** between two sentences using **pretrained FastText word embeddings**. Cosine similarity is used to measure how close the vector representations of the sentences are. A value close to **1** means high similarity.



Code: 2

Image Captioning using AlexNet

```
% Clear environment
clear; clc;

% Load an example image for object recognition
img = imread('plane.jpg'); % Replace with your own image
imshow(img);
title('Input Image');

% Load AlexNet pretrained CNN
net = alexnet;

% Resize image to match the required input size of AlexNet
imgResized = imresize(img, net.Layers(1).InputSize(1:2));

% Extract features from the 'fc7' layer (fully connected)
featureLayer = 'fc7';
features = activations(net, imgResized, featureLayer, 'OutputAs', 'rows');

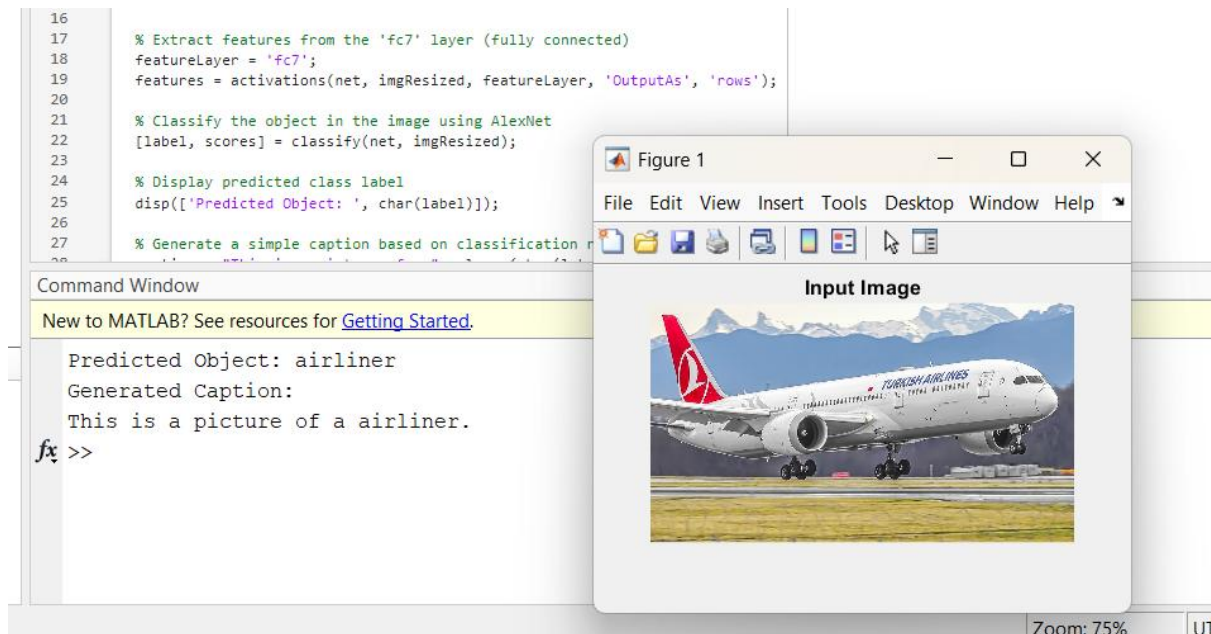
% Classify the object in the image using AlexNet
[label, scores] = classify(net, imgResized);

% Display predicted class label
disp(['Predicted Object: ', char(label)]);

% Generate a simple caption based on classification result
caption = "This is a picture of a " + lower(char(label)) + ".";

% Display the caption
disp("Generated Caption:");
disp(caption);
```

This section uses **AlexNet**, a pretrained deep CNN, to classify the main object in an image (e.g., airplane, dog). Then it uses that label to generate a **basic image caption**, simulating image captioning in an interpretable way.



Code: 3

Scene Text Detection and Translation

```

% Load another image containing scene text
img = imread('street.jpg'); % Use any image with readable text
imshow(img);
title("Scene Text Image");

% Perform OCR (Optical Character Recognition) to extract text from the image
ocrResult = ocr(img);
detectedText = upper(strtrim(ocrResult.Text)); % Convert to uppercase and remove extra spaces
disp("Detected Text:");
disp(detectedText);

% Define a translation dictionary (English → French)
dictionary = containers.Map;
dictionary("STOP") = "ARRÊT";
dictionary("SPEED LIMIT") = "LIMITATION DE VITESSE";
dictionary("YIELD") = "CÉDEZ LE PASSAGE";
dictionary("NO PARKING") = "STATIONNEMENT INTERDIT";
dictionary("ONE WAY") = "SENS UNIQUE";
dictionary("DO NOT ENTER") = "ENTRÉE INTERDITE";
dictionary("STREET") = "RUE"; % Make sure to use uppercase to match OCR

% Initialize default translation
translatedText = "Traduction indisponible."; % Default fallback

% Iterate through dictionary keys and perform replacement if match is found
keys = dictionary.keys;
for i = 1:length(keys)
    if contains(detectedText, keys{i})
        % Replace English term with corresponding French translation

```

```

        translatedText = strrep(detectedText, keys{i}, dictionary(keys{i}));
    end
end

```

```

% Display the translated text
disp("Translated Text (French):");
disp(translatedText);

```

This part performs **OCR (Optical Character Recognition)** to extract printed text from an image (like street signs), and simulates language translation using a **hardcoded English-to-French dictionary**. No internet or paid API is needed perfect for offline academic use.

Example 2: Multi-language dictionary support

```

% clear; clc;
img = imread('stop.jpg'); % Replace with your own image
imshow(img); title("Road Sign");

```

```

ocrText = ocr(img);
text = upper(strtrim(ocrText.Text));
disp("Detected Text:"); disp(text);

```

```

% Multi-language dictionary
dictFR = containers.Map("STOP", "ARRÊT");
dictDE = containers.Map("STOP", "HALT");

```

```

if isKey(dictFR, text)
    disp("French: " + dictFR(text));
end
if isKey(dictDE, text)
    disp("German: " + dictDE(text));
end

```

Example 3: OCR + fuzzy keyword matching

```

% clear; clc;
img = imread('speed.jpg');
imshow(img); title('Traffic Sign');

```

```

ocrText = ocr(img);
text = upper(strtrim(ocrText.Text));
disp("Detected Text: " + text);

```

```

% Translate based on partial keywords
if contains(text, "PARK")
    disp("Translation: STATIONNEMENT INTERDIT (No Parking)");
elseif contains(text, "SPEED")
    disp("Translation: LIMITATION DE VITESSE (Speed Limit)");
else
    disp("Translation: Not found.");
end

```

Example 4: OCR with ROI selection for better accuracy

```
clear; clc;
img = imread('stop.jpg');
imshow(img); title("Select Region for OCR");

% Manually select region
roi = drawrectangle;
pause(1); % Wait for user to draw ROI

ocrRes = ocr(img, roi.Position);
text = upper(strtrim(ocrRes.Text));
disp("Detected Text: " + text);

% Translate
dict = containers.Map("NO PARKING", "STATIONNEMENT INTERDIT");
if isKey(dict, text)
    disp("Translated: " + dict(text));
else
    disp("Translation not found.");
end
```

